THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

# Week 3: Digital security

MY472: Data for Data Scientists

https://lse-my472.github.io/

Autumn Term 2026

**Ryan Hübert**

Associate Professor of Methodology

Slides last updated: 13 October 2026

# Outline

1 Data security

2 Hashing

3 Encryption

4 Authorisation and data access

5 Authentication

6 What should you do?

# Data security

**Data security**: who can read, change or destroy data

- Applies to any data: paper records, interviews, spreadsheets

When you get data from other people, you have an ethical obligation to store it and transfer it securely

- Especially restricted data, or personally identifiable information
- You must take *proactive* steps to prevent unauthorised access
- You may need to protect sensitive data in your future job

In academic contexts (e.g., as an LSE student), you *cannot* collect sensitive data without ethics approval (LSE information)

In the UK and most of Europe, you are bound by laws like GDPR (General Data Protection Regulation)

# Who are you protecting data from?

Many, many reasons to care about data security, e.g.

- **Malicious actors** are trying to steal your money and your data
- **Governments** may spy on you, or limit strong encryption
  - You may be required to share your data at a border
- **Companies** might be accessing your data
  - LLM providers are opaque about how they use your data
  - Most cloud storage providers can read your private files
- **You**: a mistake can lead to bad consequences
  - Your devices hacked and your data stolen
  - Unintended billing charges

You should probably be more worried about this than you are

# Digital security

*Digital* security is about data security for digital data

This matters because a lot of digital data is sensitive

Three core principles guide today's lecture:

1. Data should be *protected* when it is stored ("at rest") or being transferred ("in transit") (this is **encryption**)
2. Sensitive data should only be accessed by people who are *authorised* to access it (this is **permissions**)
3. Since data access depends on identity, a person should have to *prove* who they are (this is **authentication**)

We'll cover each of these

# Very important starting caveats

1. You rarely eliminate a risk: size it and decide what to accept
   - MY472 teaches core concepts to help you evaluate risks
2. You are not a cybersecurity expert, nor am I
   - You should use standard tools that have been vetted
   - You should pay for products that provide better security
3. You need to be proactive, careful and suspicious at all times
   - Most security failures are caused by careless humans
   - Risks are increasing: pay attention to your devices and data

This lecture: core conceptual material with (macOS) examples

- ▷ Implementation details vary across operating systems/computers, but concepts apply to all machines!
- ▷ Seminars have more targeted guidance

# Outline


1 Data security

2 Hashing

3 Encryption

4 Authorisation and data access

5 Authentication

6 What should you do?

# Hash functions

A **hash function** turns any amount of digital data into a short, fixed-size **hash** (or **digest**)

A good hash function is:

- Deterministic: the same bytes always give the same hash
- Sensitive: change one bit and the hash changes completely
- One-way: you cannot recover the data from its hash

Why do this? Gives you a “fingerprint” of the bytes so you can, for example, check two files are byte-identical

Very common example: **SHA-256**, which always produces 256 bits (64 hex codes) for any digital data run through the algorithm

- We won’t explore mathematical details of hashing functions

# Hashing in Python

The package `hashlib` hashes bytes:

```
import hashlib
hashlib.sha256("LSE".encode("utf-8")).hexdigest()
```

```
'b8ec997fafd31d2dfbac31c149e7760117d147fe3369b866a248176a9db17fc3'
```

```
hashlib.sha256("LSe".encode("utf-8")).hexdigest()
```

```
'1c30077df9e58763c74d7e4a2e71e5c73957e6e6827a63b776345661d28bef3a'
```

- One changed character, a completely different hash

A file is bytes too (obviously), so you can hash any file:

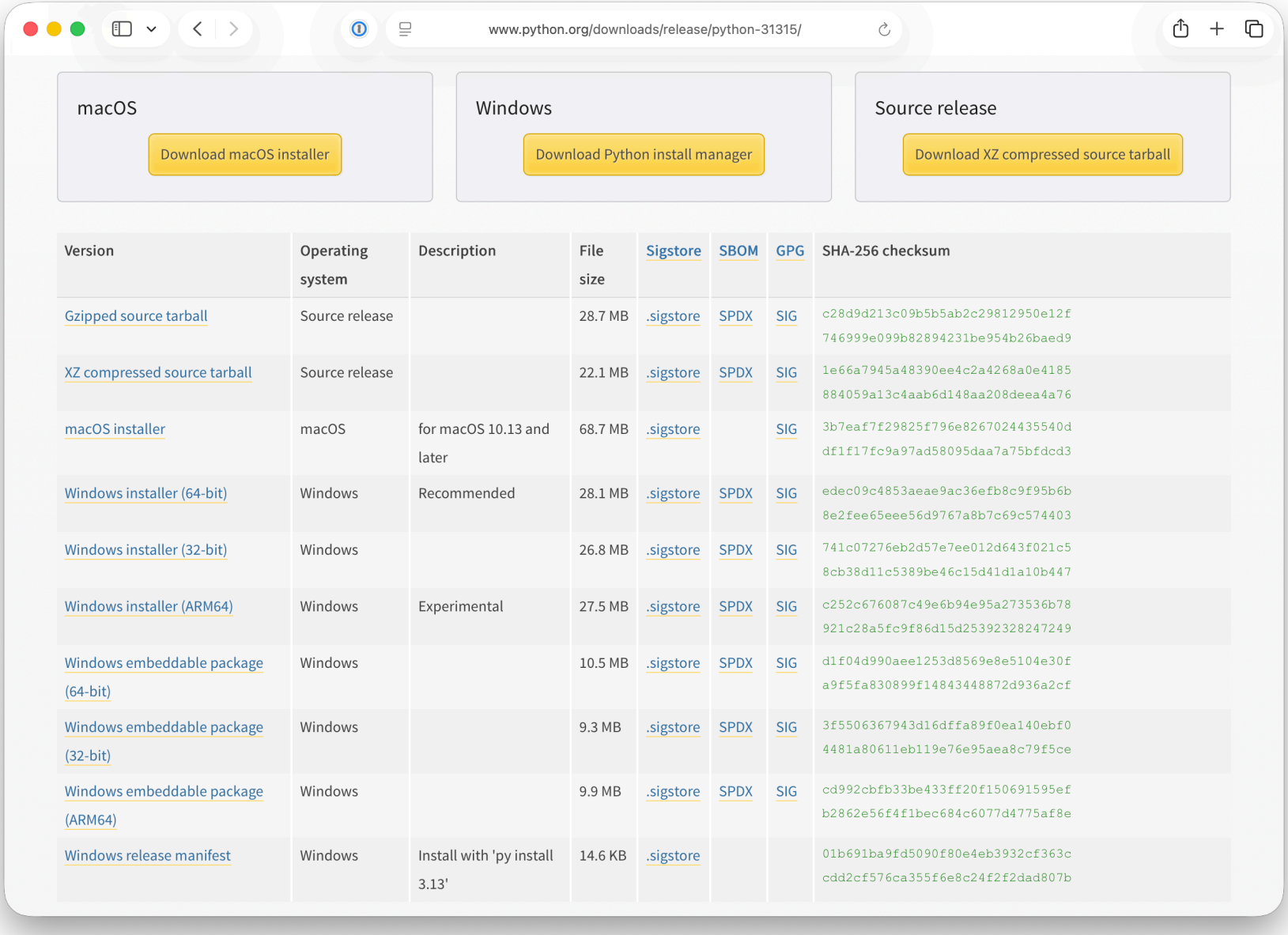
```
with open(my_path, mode = "rb") as f:
  file_hash = hashlib.sha256(f.read()).hexdigest()
file_hash
```

```
'1e7d8f50c0354032744e59fce6b4c87a3779a53b4c2813e93bef8c8c2d44db26'
```

# Uses of hashing

1. Hashing an identifier (an email, a student number) makes an ID number for a dataset, but it is not anonymous
2. Passwords are stored as hashes to reduce exposure risk
3. Version tracking software and cloud syncing services compares hashes, not whole files
    - A Git commit ID is a hash that includes the previous commit's ID, preventing quiet changes to history
    - Dropbox, iCloud, etc.: hash each file (or piece of a file) to find what changed, and send only changes
4. **Checksums** are hashes used to check data arrived intact
    - Software download pages often publish checksums
    - Hash your copy: if it matches, not one byte changed

# Checksums




| Version | Operating system | Description | File size | Sigstore | SBOM | GPG | SHA-256 checksum |
|---|---|---|---|---|---|---|---|
| Gzipped source tarball | Source release | | 28.7 MB | .sigstore | SPDX | SIG | c28d9d213c09b5b5ab2c29812950e12f746999e099b82894231be954b26baed9 |
| XZ compressed source tarball | Source release | | 22.1 MB | .sigstore | SPDX | SIG | 1e66a7945a48390ee4c2a4268a0e4185884059a13c4aab6d148aa208deea4a76 |
| macOS installer | macOS | for macOS 10.13 and later | 68.7 MB | .sigstore | | SIG | 3b7eaf7f29825f796e8267024435540ddf1f17fc9a97ad58095daa7a75bfdcd3 |
| Windows installer (64-bit) | Windows | Recommended | 28.1 MB | .sigstore | SPDX | SIG | edec09c4853aeae9ac36efb8c9f95b6b8e2fee65eee56d9767a8b7c69c574403 |
| Windows installer (32-bit) | Windows | | 26.8 MB | .sigstore | SPDX | SIG | 741c07276eb2d57e7ee012d643f021c58cb38d11c5389be46c15d41d1a10b447 |
| Windows installer (ARM64) | Windows | Experimental | 27.5 MB | .sigstore | SPDX | SIG | c252c676087c49e6b94e95a273536b78921c28a5fc9f86d15d25392328247249 |
| Windows embeddable package (64-bit) | Windows | | 10.5 MB | .sigstore | SPDX | SIG | d1f04d990aee1253d8569e8e5104e30fa9f5fa830899f14843448872d936a2cf |
| Windows embeddable package (32-bit) | Windows | | 9.3 MB | .sigstore | SPDX | SIG | 3f5506367943d16dffa89f0ea140ebf04481a80611eb119e76e95aea8c79f5ce |
| Windows embeddable package (ARM64) | Windows | | 9.9 MB | .sigstore | SPDX | SIG | cd992cbfb33be433ff20f150691595efb2862e56f4f1bec684c6077d4775af8e |
| Windows release manifest | Windows | Install with 'py install 3.13' | 14.6 KB | .sigstore | | | 01b691ba9fd5090f80e4eb3932cf363ccdd2cf576ca355f6e8c24f2f2dad807b |

# Checksums

```
import os
from urllib.request import urlretrieve

py_checksum =
↪  "3b7eaf7f29825f796e8267024435540ddf1f17fc9a97ad58095daa7a75bfdcd3"

# Download to Downloads folder
ddir = os.path.expanduser("~/Downloads")
rfile = "https://www.python.org/ftp/python/3.13.15/python-3.13.15-macos11.pkg"
lfile = os.path.join(ddir, rfile.split("/")[-1])
if not os.path.exists(lfile):
  urlretrieve(rfile, lfile)

# Read and hash bytes, then compare to checksum
with open(lfile, mode = "rb") as f:
  py_hash = hashlib.sha256(f.read()).hexdigest()
print(py_hash)
```

```
3b7eaf7f29825f796e8267024435540ddf1f17fc9a97ad58095daa7a75bfdcd3
```

```
print(py_checksum == py_hash) # should be True
```

```
True
```

# Outline


1 Data security

2 Hashing

3 Encryption

4 Authorisation and data access

5 Authentication

6 What should you do?

# Encryption

**Encryption** is the process by which “meaningful” digital data is transformed into a format that is “unmeaningful”

- In other words, encryption “locks” your digital data

When you **encrypt** digital data:

- Bytes are changed using an **cipher key**
- The resulting sequence of (changed) bytes is called **ciphertext** and it is meaningless on its own (i.e., without the cipher key)

When you **decrypt** digital data:

- The cipher key is used to change ciphertext bytes into the original (unencrypted) bytes

Note: you can encrypt *any* digital data, not just text

# Cipher keys

Cipher keys are digital data, and all digital data is bytes

- A cipher key is sequence of *randomly generated* bytes
- More bytes $\rightarrow$ more secure, since hackers need more guesses
- Key with 1 byte: can be guessed in less than 256 tries ($2^8$)

Modern standard is **256-bit AES**, an algorithm that uses keys with 32 random bytes ($8 \times 32 = 256$ random bits)

- Stands for Advanced Encryption Standard
- nearly impossible to break by "brute force"
- 256-bit AES key can be guessed in less than $2^{256}$ tries (this is a HUGE number)

# Encrypting and decrypting data in Python

The AES-GCM encryption scheme (implemented in the `cryptography` package, see the docs) takes three inputs:

```
import os
secret = "Hello!".encode("utf-8")   # 1. the data, as bytes
# secret = b"Hello!"                # <- equivalently
key = os.urandom(32)                # 2. a cipher key
iv = os.urandom(12)                 # 3. an initialisation vector
print(key.hex(" "))   # danger -- this is a secret!
```

```
33 01 a3 e8 57 2d 15 80 07 79 0b 0c d2 13 e0 c5 8e 6d 33 65 c5 96 50
06 bd 00 b7 3b eb 90 43 0a
```

```
print(iv.hex(" "))    # no danger -- this is not secret!
```

```
40 7e 90 86 c5 a4 68 7a ef 56 2f 92
```

**Initialisation vector** (IV) is not secret, but needed to decrypt

- Stored with the ciphertext, as its first bytes

# Encrypting and decrypting data in Python

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
print(secret.hex(" "))       # original
```

```
48 65 6c 6c 6f 21
```

```
secret_enc = AESGCM(key).encrypt(iv, secret, None)
print(secret_enc.hex(" "))   # ciphertext and 16-byte "tag"
```

```
13 51 5b d4 ad 8f 0d 10 af 2e 45 4f 86 43 74 64 17 7f ec 05 00 d0
```

- The 16-byte tag protects against tampering

Conventional to save encrypted data as IV + ciphertext + tag:

```
# Save directly as raw bytes
with open("encrypted_data.bin", "wb") as f:
    f.write(iv + secret_enc)
# Or as hex codes, saved in plain text
with open("encrypted_data.txt", "w") as f:
    f.write((iv + secret_enc).hex())
```

# Encrypting and decrypting data in Python

To decrypt, you need the key and the IV and the tag:

```
# Read the data from disk
with open("encrypted_data.txt", "r") as f:
     encrypted_data = f.read()

# Convert from hex string to raw bytes
encrypted_data = bytes.fromhex(encrypted_data)

# Extract the IV and the ciphertext (with tag)
iv = encrypted_data[0:12]
secret_enc = encrypted_data[12:]

# Decrypt
AESGCM(key).decrypt(iv, secret_enc, None).decode("utf-8")
```

```
'Hello!'
```

# Why do you need an IV for encryption?

Encrypt “MY472” twice, with the same key and same IV:

```
AESGCM(key).encrypt(iv, b"MY472", None).hex(" ")
```

```
'16 6d 03 8f f0 6b 16 a9 b8 61 4c e1 4e 3a 4e 04 66 23 82 c7 58'
```

```
AESGCM(key).encrypt(iv, b"MY472", None).hex(" ")
```

```
'16 6d 03 8f f0 6b 16 a9 b8 61 4c e1 4e 3a 4e 04 66 23 82 c7 58'
```

This is a security risk: without an IV (or with same one), someone who sees both could tell you sent the same message twice

So use *different* IV each time to get completely different ciphertext

```
AESGCM(key).encrypt(os.urandom(12), b"MY472", None).hex(" ")
```

```
'ca db b0 eb 5b a6 79 09 5e a5 f4 5e 73 e7 78 ac df a2 4c 9e 89'
```

```
AESGCM(key).encrypt(os.urandom(12), b"MY472", None).hex(" ")
```

```
'6a bf 70 68 06 2e 25 52 19 7e 8d c0 f6 60 8b dd 80 2e 64 fc ad'
```

# Sharing encrypted data

You just saw **symmetric encryption** where one cipher key encrypts *and* decrypts

If you want multiple people to be able to encrypt and/or decrypt, you need to (securely) share the key, but how?

- Meet in person: works, but not practical
- Sending the key over a network is risky

How can we be sure nobody intercepted a cipher key we sent?

- Don’t send it in the first place

# Key pairs and key agreement

The idea: each person makes a **key pair**

- A **private key** that you keep and never send
- A **public key** that you can give to anyone, and that *cannot be used to work out the private key*

Person A and Person B swap public keys. Then:

- A's private key + B's public key $\rightarrow$ a **shared secret**
- B's private key + A's public key $\rightarrow$ the *same* shared secret

This is **key agreement**: both can create same secret on their own machines, and nothing secret was sent over a network

- Shared secret becomes cipher key, private keys not shared
- Key pairs are also used to prove who you are (later)

# Key agreement in Python

Two people can make key pairs with `cryptography`, using X25519 (built on Curve25519):

```
from cryptography.hazmat.primitives.asymmetric import x25519
ann_key = x25519.X25519PrivateKey.generate() # private key
ann_public = ann_key.public_key()            # public (from private)
bob_key = x25519.X25519PrivateKey.generate() # private key
bob_public = bob_key.public_key()            # public (from private)
```

They swap their public keys, make a shared secret that can be used as a cipher key for symmetric encryption of sensitive data

```
shared_ann = ann_key.exchange(bob_public)    # Ann's shared secret
shared_bob = bob_key.exchange(ann_public)    # Bob's shared secret
shared_ann == shared_bob                     # <- they're the same!
```

```
True
```

# Data at rest and in transit

Data is **at rest** on a storage device, or **in transit** over a network

At rest, your data sits on a disk you control

- One person, one cipher key, so symmetric encryption is enough
- Encrypt your whole disk (FileVault or BitLocker)

In transit, data moves between programs on different computers

- A **client** is a program that makes a request, and a **server** is a program (usually on another computer) that answers (week 8)
- WhatsApp on a device is a client, so a message (data) travels: client $\rightarrow$ server $\rightarrow$ client

How is data encrypted in transit so it remains private while moving?

# Encrypting data in transit

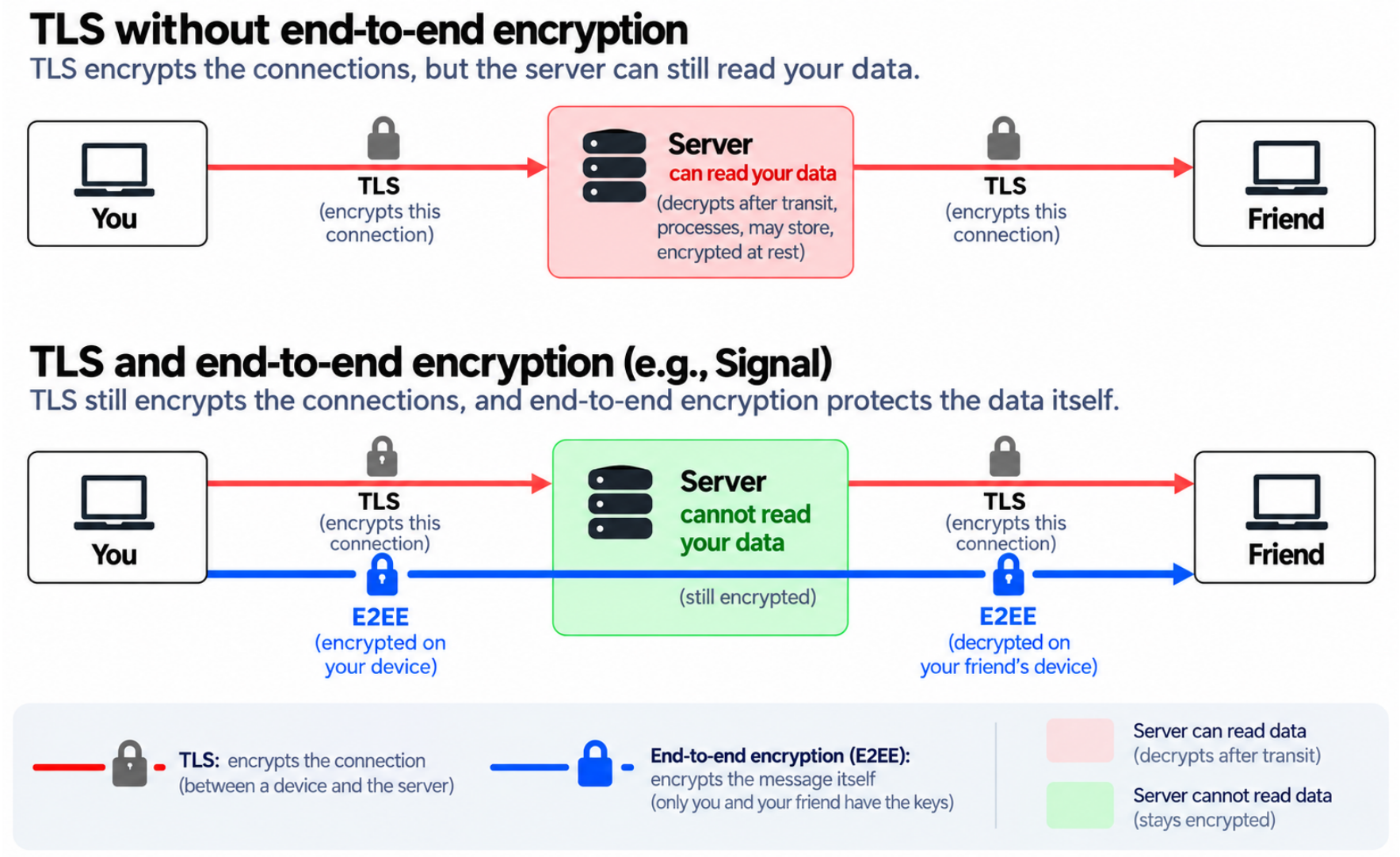
Today, encryption in transit uses the basic ideas we've developed:

- key agreement → shared secret → AES-GCM

The difference is which two parties do the key agreement

1. **TLS** (Transport Layer Security): the client and the server
   - Server decrypts data on arrival, so it can read and store it
   - `https://` means a website uses TLS
2. **End-to-end encryption (E2EE)**: the two clients
   - The server only relays ciphertext, and holds no key for it
   - E2EE: Signal, WhatsApp, Apple's Advanced Data Protection (not on UK accounts)
   - Not E2EE: most consumer cloud storage, email

# Understanding where data is decrypted




Source: ChatGPT 6

# Outline

1 Data security

2 Hashing

3 Encryption

4 Authorisation and data access

5 Authentication

6 What should you do?

# When encryption stops helping

Encryption protects data you are not *actively* using

- To work with data, you have to decrypt it
- When you log in to a device, your files are decrypted for every app you open and script you run: your browser, Positron, Dropbox, Python code from the internet, etc.

A risk because your laptop holds research data, email, downloads, and programs from many sources, all decrypted together

So, most of the time your data is protected by rules about which *users* may access it

- These rules are known as **permissions**

# Processes and the kernel

A **process** is a running program, with its own private memory

- The *program* is a file on disk; a *process* is that program running
- Can view active processes at the command line with `ps` on macOS/Linux or `Get-Process` in Powershell
- GUIs: Activity Monitor (macOS), Task Manager (Windows)

A process cannot use hardware itself (disk, network, etc.)

- Only the **kernel** has direct access to hardware
- The kernel is the core of the operating system
- A process has to ask the kernel to read data, use network, etc.

The kernel answers yes or no, but how does it decide?

# Every process runs as a user

Every process runs as a **user**: an account the kernel knows

- The processes you start run as the account you logged in as

Some accounts can do more:

- **root** (the **superuser**) is not bound by the permission rules
- An **administrator** is an ordinary user allowed to run a command as root by re-entering their own password

Kernel sees processes and their users, and usually decides access based on the user, not the program

```
$ ps -ax -o user,pid,comm     # user, process ID, program
USER               PID  COMM
root                 1  /sbin/launchd
_windowserver      412  /System/Library/.../WindowServer
rhubert            905  /Applications/Positron.app/.../Positron
rhubert           1288  /Applications/Dropbox.app/.../Dropbox
```

# Permissions on files and folders

On macOS/Linux: files have owners, groups and permissions

- Three sets of rules: one for the owner, one for the group, one for everyone else
- The kernel checks them on every request
- View them with `ls -l <path>` and change them with `chmod` (only owner or root can do this)

Find your user and group memberships:

- Who am I? `id`
- Am I an administrator? `id -Gn` (look for `admin`, `sudo` or `wheel`)

Slightly different implementation on Windows (seminar)

# Permissions on files and folders

```
$ mkdir demo
$ touch demo/test.txt
$ ls -l demo/test.txt
-rw-r--r--  1 rhubert  staff  0 Oct 12 13:05 demo/test.txt
$ chmod 444 demo/test.txt
$ echo hi > demo/test.txt
zsh: permission denied: demo/test.txt
$ ls -l demo/test.txt
-r--r--r--  1 rhubert  staff  0 Oct 12 13:08 demo/test.txt
```

# sudo

Command `sudo` runs another command with elevated privileges

- Available on macOS/Linux and Windows 11 (but different)
- Older versions of Windows: use “Run as administrator”

For example in Terminal on macOS:

```
$ cat /etc/sudoers
cat: /etc/sudoers: Permission denied
$ sudo cat /etc/sudoers
Password:
#
# Sample /etc/sudoers file.
...
```

A command run with `sudo` can access or change files normally protected — **this is very risky**

# What a process running as you can do

You constantly run code you have never read: `pip install`, apps, shared notebooks, scripts from the internet

Such code needs no extra permission to:

- read configuration files and every file in your Dropbox
- open a network connection and send any of it somewhere
- delete any file you own

Not a bug: the code runs as your user, so the kernel approves

An **agent** is a program that runs an LLM in a loop: it reads files, runs commands and edits things on your machine

- To the kernel, the agent is just a process *you* started

# Running a command from Python

A process can start another program called a **subprocess**

In Python, use the `subprocess` package:

```
import subprocess
result = subprocess.run(["git", "--version"],
                        capture_output=True, text=True)
print(result.stdout)
```

```
git version 2.54.0 (Apple Git-157)
```

- Python is a process (running as you) that asks the kernel to start `git` as subprocess, waits, and collects its output
- The command is a list: program, then one item per argument
- `capture_output=True` keeps the output in `result`, and `text=True` decodes it from bytes to a string

# What a subprocess inherits

A subprocess runs as the same user as the process that started it

- So it can access everything your user can
- It also gets a copy of the parent's environment (later)
- Parent's self-imposed rules don't automatically carry over

Two risks:

1. Autonomous LLM agents can write scripts that run as you, extract your sensitive data, and send it somewhere else

```
result = subprocess.run(["git", "config", "--get-regexp", "url"],
                        capture_output=True, text=True)
print(result.stdout)
```

```
url.https://LSE-XXXXX-2627:‹YOUR_SECRET_TOKEN›@github.com/lse-my472/⌋
.insteadof https://github.com/lse-my472/
```

# What a subprocess inherits

2. A shell can interpret input as commands, which is called **shell injection** — an example on macOS:

```
text_to_show = "hi; git config --get-regexp url"
# No shell injection
no_injection = subprocess.run(["echo", text_to_show],
                              capture_output=True, text=True)
print(no_injection.stdout)
```

```
hi; git config --get-regexp url
```

```
# Shell injection
injection = subprocess.run("echo " + text_to_show,
                           shell=True, capture_output=True, text=True)
print(injection.stdout)
```

```
hi
url.https://LSE-XXXXX-2627:‹YOUR_SECRET_TOKEN›@github.com/lse-my472/.insteadof
https://github.com/lse-my472/
```

Always pass a *list* and avoid `shell=True`

# Rules inside a program

Programs enforce their own rules, which helps prevent mistakes

`.gitignore` tells Git which untracked files to leave out of commits:

```
$ echo "KEY=abc" > secret.env
$ echo "secret.env" >> .gitignore
$ git add secret.env
The following paths are ignored by one of your .gitignore files:
secret.env
$ git add -f secret.env                # -f (force): adds it anyway
```

These are **guardrails**: rules a program enforces on itself

- Can often bypass them within the program (e.g. here with `-f`), and they don't constrain other programs
- Coding agents work this way — you can tell Claude Code or Codex not to read certain files (and they *probably* won't)

# Rules the kernel enforces

On the other hand, file permissions are checked by the kernel *on every request*, and no program can skip the check

```
$ chmod 000 secret.env
$ cat secret.env
cat: secret.env: Permission denied
$ chmod 644 secret.env                # you own it, so you can undo it
$ cat secret.env
KEY=abc
```

These are **boundaries**: rules the kernel enforces on programs

- But the kernel only stops a *different* user
- Anything running as you can `chmod` the file back, then read it

# Limiting a program (for real)

When you want to limit a program, you have two main choices:

1. Rely on its guardrails (trusting the software)
2. Rely on boundaries (stronger, but you have to set them up correctly)

For boundaries, change the account or environment it runs in:

- The kernel decides by user, so to limit a program you change which user it runs as, or where it runs
- Its own user account: access limited by file permissions
- A **container**: own filesystem and processes, shared kernel
- A **virtual machine (VM)**: its own kernel as well

Each limits more and is more computationally costly

# Outline


1 Data security

2 Hashing

3 Encryption

4 Authorisation and data access

5 Authentication

6 What should you do?

# Authentication approaches

For authentication, a person's identity is verified via **credentials**

- Authentication is an important prerequisite for authorisation
- Authentication happens both at the device level (e.g., your computer) as well as over networks (e.g., cloud based services)

Authentication usually entails a person attempting to prove their identity to a **verifier**, which could be the person's computer, a remote server, etc.

Two main ways to authenticate:

1. Send credentials to the identity verifier
2. "Signing" using key pairs

# Sending credentials

Most common way to authenticate: send a username and password to a verifier that has stored the password as a hash

During “login” (authentication), the verifier:

1. looks up the username
2. fetches that account’s stored hash
3. hashes the password you sent
4. compares the result with the stored hash

The failure mode: an attacker gets the password

- Main risks: unauthorised access to user’s password storage system, and coaxing a user to reveal their password (**phishing**)
- Verifiers store hashes and add defences: slow hashing, salt, complex passwords

# Multifactor authentication

Most cloud services now use **multifactor authentication (MFA)**

- The idea: multiple credentials are better than one

There are different kinds of **factors**:

- something you know (a password),
- something you have (a phone, a hardware key)
- something you are (**biometrics**: a fingerprint or a face)

MFA implies multiple *kinds* of factors—e.g., not just two passwords

- Authentication requires at least two kinds of credentials

Note: SMS codes are less secure than authenticator apps (see this guidance)

# Signing

**Signing** involves authentication with key pairs

The user makes a key pair and sends the public key to the verifier. Then, to log in:

1. The verifier sends a **challenge**, a random number for this login
2. The user makes a **signature** from challenge and private key
3. The verifier checks the signature using the public key

Safer than sending credentials

- Nothing secret is ever sent
- Only the private key can make a valid signature
- A stolen signature is useless, since each challenge is new

One key pair can authenticate you to many services

# Signing in Python

```
from cryptography.hazmat.primitives.asymmetric import ed25519

# Set up: make a key pair, and give the public key to the verifier
private = ed25519.Ed25519PrivateKey.generate()
public = private.public_key()

challenge = os.urandom(32)            # verifier: random number
signature = private.sign(challenge)   # you: sign it
public.verify(signature, challenge)   # verifier: no error

public.verify(signature, os.urandom(32))  # new challenge: fails
```

```
cryptography.exceptions.InvalidSignature
```

# SSH

**SSH** ("secure shell") is a **protocol**: an agreed set of rules for how two programs exchange messages

- Run commands on other computers: servers, clusters, GitHub

SSH authenticates you by signing, with three additional features:

1. The server is checked too, since it signs with its **host key**
   - Recorded on first connection, checked on every later one
2. The connection is encrypted, using key agreement
3. The private key can be locked with a **passphrase**

On that machine you are one user among many, under its rules

# Passkeys

Many websites are moving this direction with **passkeys**

Like SSH, passkeys use signing, with the same three features:

1. The other side is checked: passkeys only work on the domain they were made for (not on phishing sites)
2. The connection is encrypted: passkeys only work with `https`
3. The private key is locked: you unlock it on your device (often with biometrics), a second layer

The key difference from passwords: who compares domains

- ▷ Passwords: you decide which site to type it into (easy to phish)
- ▷ Passkeys: software on your device decides (hard to phish)

# Scoping a credential

Where authentication meets authorisation: a credential can be given a **scope**, the set of things it allows

- You need different credentials to do different things
- Like a fire door: one stolen credential does not unlock *everything*
- Usually the verifier sets the scope; some let the user do it
- For a remote service, its software decides, not your kernel

Example: GitHub's **fine-grained personal access token** has one resource owner, set permissions and an expiry date

Your week 1 token was scoped, so a thief gets only your seminar materials (still some problems!)

# Storing credentials

So far, we've talked about the risk of exposing credentials when being sent over a network

There's another obvious risk: how the user stores and accesses them for later use

Three main approaches:

1. Try to remember them: don't, it incentivises bad behaviour
2. Store them in a file on a device: common, but vulnerable
3. Store them in a credential manager: the 2026 gold standard

# Credentials in your code

In your work, you'll need programmatic access to credentials

- E.g., many APIs require a "token" or "key" to authenticate you

**Important: never hard-code a credential, e.g.:**

```
my_password = "my password for everyone to see!"
```

Credentials are stored as text: passwords and tokens already are, and a cipher key is bytes, so save `key.hex()` and read back with `bytes.fromhex()`

There are two main approaches we'll cover:

1. **Environment files** hold configuration, including credentials
2. A **(secure) credential store** manages credentials for you

# Seeing your environment

When a program starts, the operating system gives it a list of named values, called its **environment**

- For example, `HOME` is the path to your home directory, and `PATH` is the list of folders searched for programs

You can see them at the command line:

```
$ echo $HOME                # macOS/Linux
/Users/rhubert
$ printenv                  # macOS/Linux: all of them
HOME=/Users/rhubert
PATH=/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin
...
$ echo $env:USERPROFILE     # Windows PowerShell
C:\Users\HUBERTR
```

Python example on macOS:

```
import os
os.environ["HOME"]
```

```
'/Users/rhubert'
```

# Environment files

The usual approach for *local* development is a `.env` file

A plain text file, saved locally, one value per line

```
USERNAME="J.Smith@lse.ac.uk"
KEY="sup3r $ecret keY!"
```

Read it with `dotenv`, then get the value you need with `os`

```
from dotenv import load_dotenv
load_dotenv("key.env")
os.environ.get("KEY")
```

```
True
```

```
'sup3r $ecret keY!'
```

Now `USERNAME` and `KEY` are **environment variables**

# The risk in an environment variable

Environment variables are not in the variables pane, and vanish when Python closes. Still, three risks

1. A process passes its environment to every program it starts
2. They can appear in logs, crash reports and error traces
3. An `.env` file sits inside a project, so it is easy to push to GitHub by mistake

Risk 1, in Python:

```
import subprocess
os.environ["MY_KEY"] = "sup3r $ecret keY!"
subprocess.run(["printenv", "MY_KEY"],
               capture_output=True, text=True).stdout
```

```
'sup3r $ecret keY!\n'
```

The subprocess inherited the key automatically!

# Keychains

More secure: use your computer's "keychain"

- A **keychain** is operating system software that stores sensitive information (like credentials) for use by other software
- Not the same thing as a password manager, which is meant for human (not software) use
- macOS: "Keychain Access". Windows: "Credential Manager"

Can save a credential to the keychain from Python using `keyring`

```
import keyring, getpass
keyring.set_password("my472", "example-key",
                     getpass.getpass("Enter your key: "))
```

A prompt appears in Console: enter the credential and press enter

# Getting a credential from the keychain

Then, when you need a token from your keychain:

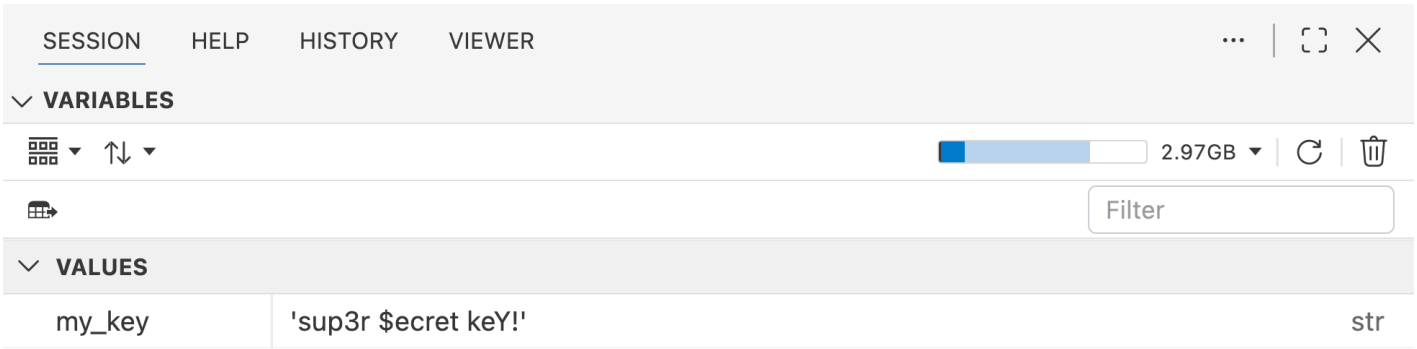
```
my_key = keyring.get_password("my472", "example-key")
```

Token is not visible in Python console/editor, unless it's printed:

```
print(my_key)
```

```
sup3r $ecret keY!
```

But, it *is* visible in Positron's variables pane (oops!):




Careful: any Python on your computer can get this key!

# Outline

1 Data security

2 Hashing

3 Encryption

4 Authorisation and data access

5 Authentication

6 What should you do?

# What should you do?

What's the take-away for you as a data scientist?

- Start by shoring up your digital security

1. **Secure your devices and accounts**
   - Use a PIN or password, biometrics, auto-lock, loss/theft settings
   - Use unique, complex passwords in a password manager and put your credentials in the keychain
   - Set up MFA on all accounts, and use passkeys where possible
2. **Be very suspicious at all times**
   - Do not give passwords or tokens to *anyone*
   - Check email addresses and URLs before clicking, look for `https`
   - Do not click past warnings from your browser or device

# What should you do?

3. **Keep less**
   - Avoid storing sensitive information in unprotected files
   - To be safest: never store sensitive data off your (fully encrypted) devices
   - If you still need to use cloud services, use services provided by your institution, that have gone through security and legal reviews
   - For E2EE, use iCloud (with ADP enabled) or manually encrypt using third-party software
4. **Know the rules**
   - Spend some time reviewing policies
   - Learn security practices of your service providers
   - Check the law before travelling with encrypted data (LSE's guidelines)